# DATA VORTEX — ROUND 1 (PHASE 2)

## SQL QUERY SUBMISSION

**Theme:** Rebuilding the Social Engine

**Participant details:**

**Team Name:** Joy

**Team Size:** 1

**Team Head:** Thathi Reddy Jivan Reddy

**Selected Questions:** E3, M5, H4

**Database:** MySQL 8.0.46

**Schema:** data_vortex

### 1. EASY — E3: Average Engagement by Platform

**Qusetions:** Calculate the average likes, shares, and comments for each platform.Which platform generates the highest average total engagement?

**SQL Query:**

```
SELECT
    platform,
    COUNT(*) AS post_count,
    ROUND(AVG(CAST(likes AS DECIMAL(10,2))), 2) AS avg_likes,
    ROUND(AVG(shares), 2) AS avg_shares,
    ROUND(AVG(comments), 2) AS avg_comments,
```

```
    ROUND(
        AVG(
            CASE
                WHEN likes IS NOT NULL
                 AND shares IS NOT NULL
                 AND comments IS NOT NULL
                THEN CAST(likes AS DECIMAL(10,2))
                    + shares
                    + comments
            END
        ), 2
    ) AS avg_total_engagement
FROM posts
WHERE platform IS NOT NULL
  AND TRIM(platform) <> ''
GROUP BY platform
ORDER BY avg_total_engagement DESC;
```

***Key SQL techniques:*** *GROUP BY, AVG, COUNT, CASE, CAST, ROUND, filtering of missing platforms, and ORDER BY*

**2. MEDIUM — M5: Detect Suspicious Engagement**

**Question:** Identify posts where shares are greater than likes plus

comments, and return the top 20 by total engagement.

**SQL Query:**

```
SELECT
post_id,
user_id,
platform,
CAST(likes AS DECIMAL(10,2)) AS likes,
shares,
comments,
(
CAST(likes AS DECIMAL(10,2))
+ shares
+ comments
) AS total_engagement
FROM posts
WHERE likes IS NOT NULL
AND shares IS NOT NULL
AND comments IS NOT NULL
AND shares > CAST(likes AS DECIMAL(10,2)) + comments
ORDER BY total_engagement DESC
LIMIT 20;
```

***Key SQL techniques:*** *calculated condition, NULL filtering, numeric CAST, derived engagement measure, ORDER BY*

*DESC, and LIMIT 20.*

**3. HARD — H4: Follower to Engagement Anomaly**

**Question:** Identify users with fewer than 5,000 followers whose total post engagement falls within the top
10% of all users.

**SQL Query:**

```
WITH user_engagement AS (
SELECT
u.user_id,
u.location,
u.follower_count,
COUNT(p.post_id) AS post_count,
SUM(
CASE
WHEN p.likes IS NOT NULL
AND p.shares IS NOT NULL
AND p.comments IS NOT NULL
THEN CAST(p.likes AS DECIMAL(10,2))
+ p.shares
+ p.comments
ELSE 0
```

```
END
) AS total_engagement
FROM users u
LEFT JOIN posts p
ON u.user_id = p.user_id
GROUP BY
u.user_id,
u.location,
u.follower_count
),
ranked_users AS (
SELECT
user_id,
location,
follower_count,
post_count,
total_engagement,
CUME_DIST() OVER (
ORDER BY total_engagement DESC
) AS engagement_percentile
FROM user_engagement
)
SELECT
user_id,
location,
```

```
follower_count,
post_count,
total_engagement,
ROUND(engagement_percentile * 100, 2) AS top_percentage
FROM ranked_users
WHERE follower_count < 5000
AND engagement_percentile <= 0.10
ORDER BY total_engagement DESC;
```

***Key SQL techniques:*** *CTEs, LEFT JOIN, user-level aggregation, CASE-based engagement calculation, CUME_DIST()window function, percentile filtering, and multi-level analysis.*